From Spectrogram to Song: A Reproducible Case Study of Audio-Conditioned, Quantum-Inspired Entropy for Cohesive Lyric Design

Methods, circuit equations, source-code provenance, cadence mapping, and an evaluation protocol

Technical research report prepared for the original audio project

16 October 2026

Abstract

This case study documents a reproducible workflow that converted one 162.8-second stereo recording into spectrotemporal measurements, an eight-outcome probability distribution from a three-qubit quantum-inspired circuit, a constrained lyrical concept, and a Suno-ready lyric and performance guide. The observed tempo was 133.928571 beats per minute (BPM), and the highest-scoring chroma-template interpretation was D-sharp/E-flat minor (Pearson correlation 0.716); neither is a ground-truth transcription. Audio energy within three frequency ranges, metaphorically designated red, green, and blue, produced the normalized vector (0.875328, 0.117359, 0.007313). A pure, classically simulated state underwent Hadamard, RY, RZ, and cyclic controlled-NOT operations. Computational-basis measurement probabilities had Shannon entropy 1.622796 bits (0.540932 of the three-bit maximum). A seeded NumPy generator drew 4,096 synthetic outcomes, ranking “neon skyline” and “streetlights” as constrained imagery anchors. Lyrics titled “Bring on the Night” were then drafted using this palette and an approximate one-line-per-bar cadence plan. Examination and rerunning of the original Python scripts confirm the reported entropy result. No quantum hardware was involved, no learned optimization was performed, no sung output was experimentally evaluated, and improvement in lyric quality remains a hypothesis rather than a finding. This paper provides equations, instrument settings, prompt records, implementation details, failure modes, proposed validation tests, and executable supplementary code.

Table of Contents

1 Introduction

Songwriting from an existing instrumental requires a sequence of decisions that are related but not interchangeable. A songwriter first estimates the musical grid: how fast are the beats, how many beats make a measure, and where does the arrangement become intense, sparse, or transitional? The next question is rhetorical: what emotional narrative could make convincing use of that arrangement? Finally, the writer must transform general imagery into syllables that fit time, reinforce a hook, and give a singer enough breath to make the performance intelligible. Digital audio analysis can support the first task, but it does not independently solve the latter two. The present experiment deliberately joins measurements, a stochastic idea-selection mechanism, and human-interpretable songwriting rules without claiming that spectral frequencies directly encode a song’s semantic meaning.

The initiating request was to examine an uploaded recording by spectrogram, develop lyrics and explicit cadence instructions suitable for Suno, and create an “RGB” entropy circuit in NumPy inspired by the workflow of PennyLane. The user wanted entropy to influence lyrical cohesion but did not want the circuitry or its scientific vocabulary to appear in the lyrics. That distinction defines the artistic objective: treat mathematical computation as an upstream constraint source, not as a literal lyrical subject. The resulting text expresses a familiar but concrete narrative of emerging from someone else’s control into self-direction, using street-level light imagery and repeated end sounds to signal continuity.

In many creative-computing demonstrations, the word *entropy* is applied loosely to randomness, surprise, or novelty. Here it has an exact mathematical meaning. The system produces a probability distribution over eight three-bit outcomes, then evaluates the uncertainty of that distribution in Shannon bits. Its entropy is neither the amount of emotional information in the lyrics nor a quality score. It does not measure the semantic variety of the complete lyric. Instead, it describes the dispersion of a small selection mechanism from which a motif and rhyme palette can be chosen. This limited role is scientifically defensible, testable, and potentially useful: randomness can broaden the selection of candidate ideas, while explicit compositional constraints can prevent the story from becoming incoherent.

The label *quantum-inspired* is also used narrowly. The implementation creates complex-valued state vectors and multiplies them by unitary matrices corresponding to quantum gates. Measurement probabilities follow the squared magnitudes of amplitudes. However, everything runs on ordinary computer hardware with NumPy; sampling is by a seeded pseudorandom-number generator. There is no physical quantum source of entropy, experimental entanglement witness, quantum speedup, quantum machine-learning model, or comparison establishing an advantage over classical probabilistic baselines. PennyLane provides a familiar model of gates, wires, and measurement operations [1], but the source script does not import PennyLane.

The importance of this case lies in the visibility of its interfaces. Music features are converted to three normalized energy components; those numbers become rotation parameters; the circuit yields eight probabilities; those probabilities rank a fixed bank of images and rhyme families; the selected ideas are assembled into a repeated hook; and that hook is mapped onto an approximate musical grid. At every step, a reader can ask whether a design decision was physically measured, mathematically deduced, heuristically chosen, or made during creative drafting. A useful methods paper should keep these categories distinct. It should also preserve the source code and output records necessary to reproduce reported numbers.

This document is therefore a technical case study, not a peer-reviewed controlled experiment. It reconstructs the workflow from the original available scripts, their saved JSON results, the generated spectrogram, and the recorded lyrics and Suno style text. The entropy script has been executed again against the same audio and reproduced the saved output byte-for-byte. The document supplies real code and command lines. It does not invent a separate lyric-scoring program, any inaccessible intermediate drafting prompts, or a set of user listening-test results that were never collected.

2 Research objectives, design, and epistemic boundaries

Four operational questions organize the investigation. First, can a short set of low-cost spectrotemporal measurements provide actionable information about pacing and dramatic contrast in an unfamiliar recording? Second, can frequency-band energies parameterize a small quantum-style circuit in a stable, reproducible way? Third, can a sampled discrete motif palette help impose consistency across verse, pre-chorus, chorus, breakdown, and outro? Fourth, what evidence would be needed to claim that this procedure is better than songwriting without the entropy step?

The first two questions concern implementation and numerical validity. Their answers can be established with code, saved artifacts, and reruns. The third concerns a plausible mechanism of constrained creativity and requires careful traceability: one can show that the final text reuses selected images and end-rhyme families, but cannot from one example determine whether the circuit caused the cohesion or merely supplied ideas that the writing stage could have selected without it. The fourth requires comparative evidence not contained in this case. It is addressed as a future experimental protocol rather than presented as an achieved result.

The data set consists of one user-provided WAV file, “Punchier Pre-Chorus and Chorus (Extended) (1) (1) (1).wav.” There are no multitrack stems, reference chord chart, verified tempo annotation, transcribed vocals, ground-truth segmentation marks, audience preference scores, or commercially released Suno outputs associated with the case. The recording is 162.8 seconds long at 48,000 samples per second, with two channels. Its SHA-256 file digest is 08641bf7a137136c956179c350aa78bf8c5f369fbe3e06605fb75dddacfb7a88. This hash identifies the input bytes used to reproduce the computational findings; it is not a claim about copyright ownership or authorship of the audio.

The project has two kinds of reproducibility. **Numerical reproducibility** means that the given file and script reproduce the frequency-band proportions, circuit parameters, probabilities, and selections. The entropy rerun meets this criterion in the available runtime. **Creative reproducibility** means that a reader can regenerate the same lyrics from the code alone. This stronger criterion is not met, because the code chooses a constrained idea palette but does not implement a full lyric-generation model or deterministic text decoder. The final writing was performed through language-based drafting informed by the computed palette and audio mapping. A reader can reproduce its *constraints* precisely, but not its linguistic choices from a Python random seed alone.

One methodological safeguard deserves emphasis. The existence of an attractive final song text does not itself validate the segmentation analysis, the key estimate, the cadence fit, or the creative value of the circuit. A feature can be useful while being approximate; a lyric can be coherent for reasons unrelated to the algorithm that suggested its imagery. Accordingly, the paper uses “estimated,” “candidate,” and “suggested” when reporting tempo, key, section identity, and alignment, while reserving exact numerical language for reproducible calculations such as a normalized probability vector or a finite Shannon sum.

3 Source materials, execution environment, and provenance

The available working directory contained the original input WAV, analyze_audio.py, track_analysis.json, track_spectrogram.png, quantum_rgb_entropy.py, quantum_rgb_entropy_results.json, and Bring_On_The_Night_Suno_Lyrics_and_Cadence.md. It also contained an archive of the original song kit and a smaller JPEG preview of the spectrogram. These files show what was actually coded or recorded in the earlier workflow. The scientific manuscript is a later interpretation of those materials, not an assertion that a formal paper or statistical validation existed at the time the song was drafted.

The computing environment used for inspection and the independent entropy rerun reports Python packages NumPy 2.3.5, SciPy 1.17.0, librosa 0.11.0, SoundFile 0.13.1, and Matplotlib 3.10.8. Because NumPy's default random-bit generator can be version-dependent, recording a Python environment improves long-term reproducibility. The circuit obtains its seed from the input waveform, but bit-for-bit synthetic draws may differ if a future NumPy default implementation changes. In a strict archival version, the default generator could be replaced with an explicitly constructed np.random.Generator(np.random.PCG64(seed)) and package versions pinned. That latter line is a *recommended improvement*, not the exact line in the original source.

The source code of analyze_audio.py accepts its file path through a hard-coded string and writes directly to fixed filenames. The second program accepts a positional WAV path and an optional --json output path, making it easier to rerun. Its audio_rgb() function downsamples general input rates to 12 kHz using a greatest-common-divisor resampling ratio; the analysis script uses resample_poly(mono, 1, 4), which is specifically appropriate for the observed 48 kHz file. These implementation differences matter. A reader who substitutes a 44.1 kHz recording into the analysis script without adjustment would not obtain a correct 12 kHz output, whereas the entropy script explicitly handles that rate conversion.

Saved outputs were inspected rather than guessed from the image. The analysis record reports a global mono waveform root-mean-square (RMS) amplitude of 0.1489803 and a peak absolute sample magnitude of 0.6088131, in the file's floating-point amplitude scale. It also reports 8-second section summaries, onset and beat information, and template-based key candidates. The entropy record provides exact rounded band values, three rotation angles, all eight measurement probabilities, a Shannon entropy, a deterministic seed, and selected imagery. The reproducibility bundle includes both original scripts unmodified alongside the report and the existing data outputs.

The prompt record is narrower than a complete chain of every sentence generated during drafting. The user-facing request, the exact final Suno style field, the exact lyric field, the section tags, and the saved cohesion constraints are available. No separately logged internal prompt sequence for intermediate language-model revisions is present. Therefore, this paper can reproduce the final prompts and the computed creative constraints but cannot honestly provide a verbatim unavailable sequence of hidden intermediate drafting instructions. Section 11 explicitly labels a proposed *replication prompt* as a reconstruction rather than a recovered historical artifact.

4 Digital audio processing and the spectrogram

The waveform was loaded with the SoundFile library as a 32-bit floating-point array of shape samples by two channels. The script created a mono track by taking the arithmetic mean of the left and right channels, mono = y.mean(axis=1). Averaging is convenient for feature extraction but can partially cancel anti-phase material. The original stereo waveform was not spatially analyzed, and no mid/side, loudness-standardized, or channel-aware comparisons were performed. For this particular demonstration, a mono representation was sufficient to expose the strong rhythmic pulse and the broad arrangement changes visible in the spectrogram.

The sample rate was then reduced by a factor of four from 48 kHz to 12 kHz with polyphase resampling. Unlike naively discarding three of every four samples, scipy.signal.resample_poly includes low-pass filtering to reduce aliasing [2]. Under a 12 kHz sampling rate, the Nyquist frequency is 6 kHz. This was acceptable for the chosen frequency plot and feature bands but discards spectral content above 6 kHz. Consequently, this spectrogram cannot support conclusions about cymbal air, ultra-high-frequency content, or all audible high-frequency transients. It is a deliberately bandwidth-limited analytical representation, not a full-resolution visual inventory of the recording.

The short-time Fourier transform (STFT) of a discretely sampled signal $x[n]$, a window $w[n]$, transform size N, and frame hop h can be written as

$$X(m,k)=\sum_{n=0}^{N-1} x[n+mh]\,w[n]\exp\left(-j\frac{2\pi k n}{N}\right).$$

Here m indexes frames and k indexes frequency bins. The original code used scipy.signal.stft(x, fs=12000, nperseg=2048, noverlap=1792, boundary=None), implying a hop of 256 samples. With a 12 kHz sample rate, the window spans approximately 170.67 milliseconds and adjacent frames begin 21.33 milliseconds apart. The spacing of discrete Fourier bins is $12000/2048=5.859375$ Hz. That bin spacing should not be confused with the ability to resolve arbitrarily close physical sinusoids: the finite time window and its spectral leakage impose their own limits. The default Hann window used by SciPy trades reduced leakage against temporal and frequency smearing [3].

The plotted magnitude spectrogram takes $|X(m,k)|$ and applies a logarithmic mapping:

$$D(m,k)=20\log_{10}\left(max\left(|X(m,k)|,10^{-7}\right)\right),$$

followed by a global shift $D_{rel}(m,k)=D(m,k)-max_{m,k}D(m,k)$. Values are shown with a display range of approximately -72 to 0 dB relative to the strongest plotted STFT coefficient. The image uses Matplotlib's magma color scheme and displays 0–6 kHz on its vertical axis. The display is therefore **relative intensity**, not a calibrated sound-pressure-level measurement. Values in this image do not mean that the entire audio file has a particular acoustic decibel level, and their brightness should not be interpreted as the perceived loudness of a human listener without a more appropriate perceptual model.

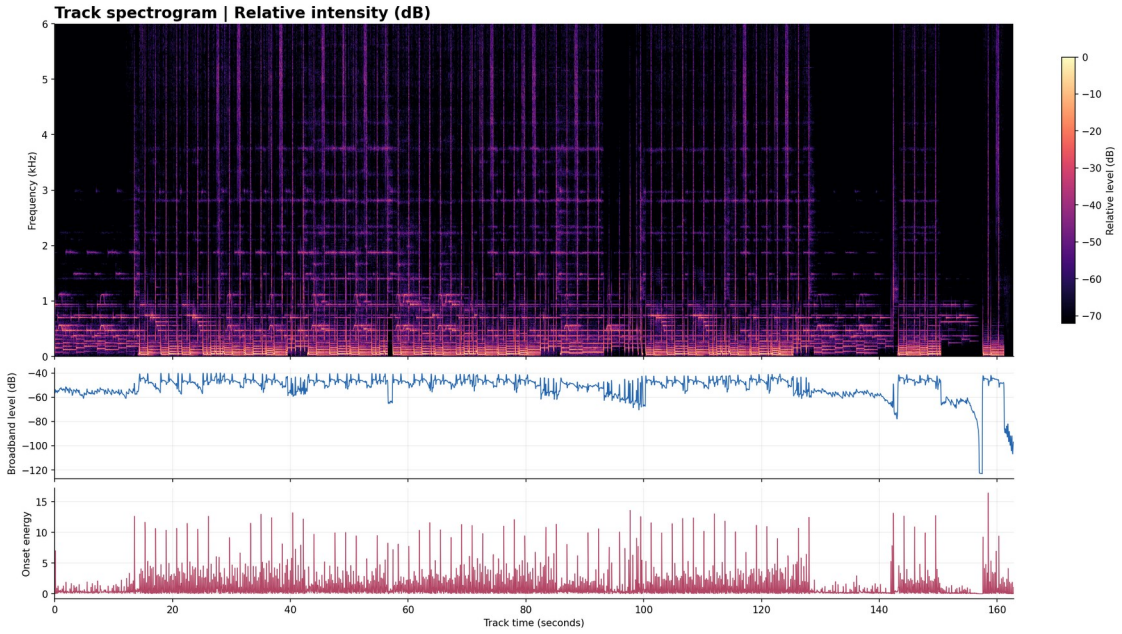


Figure 1. Original analysis plot. Top: 0–6 kHz relative-magnitude STFT spectrogram; middle: STFT-derived broadband magnitude proxy; bottom: librosa onset-strength envelope. The figure is a reproducible image generated from the uploaded WAV.

Two supporting curves were added below the spectrogram. One computes the root-mean-square of the STFT magnitude coefficients per frame, $r_m = \sqrt{K^{-1}\sum_{k=1}^{K}\left|X(m,k)\right|^2}$, then plots $20\log_{10}\left(r_m + 10^{-7}\right)$. In the original source this appears as "Broadband level (dB)." A precision point is that r_m is not the same quantity as sample-domain windowed RMS and is not independently calibrated to dBFS. The other curve is a librosa onset-strength envelope. librosa's default onset strength is derived from positive changes in a reference-filtered log-power mel representation, providing evidence of abrupt musical changes and note or percussion attacks [4]. The original code also calculates a simpler positive spectral-flux statistic from the linear STFT magnitude matrix for each eight-second block; these two flux representations should not be conflated.

An STFT plot cannot identify the exact word "chorus." It can reveal repeated textures, greater high-frequency activity, discrete attacks, silent breaks, low-frequency emphasis, and abrupt changes that are compatible with song structure. Determining formal section names requires higher-level comparison, melody, harmony, or listening. In the present project, the image and interval energy statistics guided the placement of proposed sections; the labels "verse," "pre-chorus," "chorus," and "bridge" are constructive suggestions, not the output of a trained automatic structure classifier. This distinction is essential if the technique is later published or compared with annotated structural-segmentation benchmarks [5].

5 Beat, tonal evidence, and structural interpretation

Tempo was estimated in two ways that rely on related evidence. First, librosa generated an onset-strength envelope from the resampled waveform using the same 256-sample hop. The script called librosa.feature.tempo and aggregated framewise tempo candidates by their median. Second, librosa.beat.beat_track combined onset periodicity with dynamic programming to recover beat times approximately consistent with one tempo [6]. Both reported 133.928571 BPM. The first reported beat timestamps begin at approximately 0.13, 0.58, 1.02, 1.47, 1.92, and 2.37 seconds, indicating successive separations near 0.45 seconds. These observations support a fast quarter-note pulse. They do not prove that listeners will agree about a 4/4 meter, since beat trackers normally recover periodicity without an authoritative meter transcription.

At the estimated tempo, a beat lasts $T_b = 60/133.928571 \approx 0.448$ seconds. A nominal 4/4 bar lasts $T_{bar} = 4\,T_b \approx 1.792$ seconds, and eight such bars last about 14.336 seconds. These values were used to create a convenient prospective songwriting grid. The section grid begins with a nominal eight-bar introduction and assumes a regular continuation of measures thereafter. Its zero point is an editorial choice; a verified musical downbeat would need beat-phase inspection and, ideally, listening. Therefore, the neat eight-bar times in the final guide are mathematically fitted to the approximate tempo, not independently discovered exact structure boundaries.

The chroma calculation begins from the STFT magnitude and calls librosa.feature.chroma_stft(S=(mag*1000)**2, sr=12000, hop_length=256, n_fft=2048). Chroma folds pitches across octaves into twelve pitch-class bins: C, C-sharp, D, and so on through B. This representation can characterize harmonic pitch content in a way that is partially invariant to octave, but its interpretation is complicated by percussion, tuning, timbre, harmonic overtones, and changes of chord root [7]. The analysis script averaged each chroma bin over the full recording and compared the resulting twelve-element vector with rotated standard major and minor key profiles.

For each candidate tonic r and tonal mode s, the score was Pearson's correlation $\rho\left(\acute{c}, roll\left(p_s, r\right)\right)$, where $\acute{c}$ is the mean chroma vector and p_s is a fixed twelve-value tonal profile. The highest entry was D-sharp minor, equivalent in equal temperament to E-flat minor, with correlation 0.716. The second entry was F-sharp major at 0.635, and the third was A-sharp minor at 0.596. These competing interpretations are musically meaningful rather than mere numerical noise: E-flat minor and F-sharp major share a relative-major/minor relationship, so a mix can contain pitch classes compatible with both. This scoring technique is based on tonal-profile comparisons, related to key-perception research by Krumhansl and Kessler [8]. The correlation is a similarity score, **not** an estimated posterior probability that the track is definitively in a given key.

Because the original goal concerned vocal construction rather than harmonic transcription, the candidate minor-key reading influenced the descriptive Suno style as "minor-key alternative electro-pop" without prescribing exact chord tones or an exhaustive sung melody. A more robust production workflow would use harmonic-percussive source separation, chroma smoothing, segmentation by chord region, manual chord checks, and perhaps a key-confidence

measure across several chroma estimators. None of those extensions were implemented in the existing scripts; introducing them into a claimed historical reproduction would be misleading.

For structural contrast, the program computed the sample-domain RMS separately for successive eight-second intervals. Relevant values include 0.0656 in seconds 0–8, 0.1150 in seconds 8–16, 0.1638 in seconds 16–24, 0.1878 in seconds 24–32, 0.0916 in seconds 88–96, 0.0567 in seconds 128–136, and 0.1694 in seconds 144–152. These numbers support three broad impressions visible in Figure 1: a restrained beginning moving into a stronger early arrangement, an intermediate reduced-energy stretch, and a sharper drop around 128 seconds followed by a renewed rise near 144 seconds. RMS is a rough energy proxy; quiet passages can still carry emotionally intense harmony or foreground vocals. The report refrains from equating amplitude with musical importance.

The ranking of changes in eight-second RMS intervals placed the largest absolute decrease at 128 seconds, with a change of about 0.1075 in the RMS statistic. The largest subsequent increase appeared at 144 seconds, about 0.0816. Earlier notable changes occurred at 8, 16, 80, 88, 96, and 104 seconds. These boundary estimates are quantized to the eight-second aggregation grid and must not be read as precise splice-point detections. The displayed spectrogram contains a more continuous portrayal of the changes, but even a high-resolution view is not a substitute for inspecting musical bars around each transition.

The lyric arrangement uses the following editorial projection. A 0:00–0:14.3 introduction leads into a sixteen-bar verse; an eight-bar pre-chorus runs approximately 0:43.0–0:57.3; a sixteen-bar chorus follows to about 1:26.0; a breakdown spans to 1:40.3; another eight-bar build and eight-bar chorus occupy the next interval; then a bridge, reprise, outro, and final tail complete the record. This grid explains how the writer could map a repeated hook to a 162.8-second file. It should not be interpreted as a statement that the input instrumental’s original composer marked the same functional sections. Indeed, some intended vocal sections cross weaker or stronger energy boundaries. In a final studio production those alignments should be auditioned and revised, not trusted solely because their arithmetic is neat.

Time in recording	Proposed writing section	Approximate bars	Intended delivery
0:00–0:14.3	Introduction	8	Instrumental rise, minimal distant vocal
0:14.3–0:43.0	Verse 1	16	Rhythmic, close and relatively dry
0:43.0–0:57.3	Pre-chorus 1	8	Space at first; increasing urgency
0:57.3–1:26.0	Chorus 1	16	Repeated eight-bar hook; vocal doubles
1:26.0–1:40.3	Breakdown	8	Wordless texture, then intimate voice
1:40.3–1:54.6	Pre-chorus 2	8	Rebuild tension and anticipation

Time in recording	Proposed writing section	Approximate bars	Intended delivery
1:54.6–2:08.9	Chorus 2	8	Full hook reprise
2:08.9–2:23.3	Bridge	8	Half-time feel, gradual increase
2:23.3–2:30.5	Final hook	4	Peak emphatic delivery
2:30.5–2:42.8	Outro and tail	~7	Softer closure and reverb decay

6 The RGB metaphor: conditioning a probability model on sound

The term RGB usually denotes red, green, and blue color components. In this project it is a conceptual label for **three spectral-energy bands**, not a measurement of colors or a physical relationship between light and music. “Red” represented 40–250 Hz, “green” represented 250–2,000 Hz, and “blue” represented 2,000–6,000 Hz. The colors make a three-component control vector easy to remember and permit direct assignment to three simulated qubits. The selection of these bands was a design choice; no evidence was collected that this three-way split is optimal for lyrics. Alternative band definitions could change the state probabilities and hence the chosen imagery.

The entropy program read the same WAV, averaged stereo channels, and computed an audio-derived random seed **before** resampling. It selected every 257th sample of that original mono float32 sequence, converted the resulting values to bytes, computed SHA-256, took the first eight digest bytes, and interpreted those bytes as a little-endian unsigned integer. The resulting seed was 14288464744916120875. This procedure binds the sequence of pseudorandom draws to the input waveform. If the waveform bytes or the downmix change, the seed can change even when a human listener perceives the tracks as essentially identical. Thus it is a reproducibility device and a simple audio fingerprint, not a perceptually invariant music hash or a cryptographic source of randomness.

For frequency analysis, the program resampled to 12,000 Hz if necessary and chose 128 starting positions evenly across the waveform. At each position it extracted an 8,192-sample frame, multiplied by a Hann window, calculated the real-valued fast Fourier transform, and accumulated squared magnitudes. Each frame spans approximately 0.6827 seconds. The spacing between frame starts is substantially longer than the spectrogram’s 21.33 ms hop, so these are two deliberately distinct resolutions: a dense STFT for visual and onset-related analysis, and sparse long-window FFTs for broadband energy ratios. The accumulated magnitude squares were not normalized to physical watts or sound-pressure units, but the ratio within the same calculation is appropriate for a relative feature vector.

If $F_j[k)$ denotes the Fourier transform of windowed frame j, the accumulated energy for band b was

$$E_b = \sum_{j=1}^{128} ❑ \sum_{k : f_k \in B_b} \left| F_j[k] \right|^2, b \in \{R, G, B\}.$$

The normalized vector is $r_b = E_b / (E_R + E_G + E_B)$. This maps a nonnegative energy triple to a probability-simplex-like vector satisfying $r_R + r_G + r_B = 1$, provided the selected audio contains nonzero energy in at least one band. The implementation does not have a guard for an entirely silent input, so a completely silent file would produce a zero denominator and invalid values. This is an actionable robustness issue rather than a hidden property of quantum theory.

For the uploaded track, the vector was $r = (0.87532849, 0.11735883, 0.00731268)$ before six-decimal rounding. Roughly 87.5 percent of the accumulated energy in the specified 40–6,000 Hz ranges came from the low band. That does **not** mean 87.5 percent of perceived loudness was bass, since perceived loudness is frequency-weighted, and neither the FFT procedure nor the energy sum applied a psychoacoustic equal-loudness model. It does suggest that the arrangement has substantial low-frequency power relative to the other two arbitrarily defined bands. The program uses that imbalance as numeric material for rotation gates instead of directly verbalizing it as “darkness,” “bass,” or an emotional sentiment.

One could compute the ordinary Shannon entropy of the normalized band vector itself, $H(r) = -\sum_b r_b \log_2 r_b$. For these values the result is about 0.5828 bits, with a theoretical three-symbol maximum of $\log_2 3 \approx 1.585$ bits. The actual system did **not** use this quantity as its reported “quantum RGB entropy.” It used the normalized bands to parameterize a circuit, then computed a different eight-outcome entropy. Keeping these two quantities separate avoids a common methodological error. The input distribution is over three labeled frequency groups; the output distribution is over the computational basis states of three qubits. They have different state spaces, denominators, maxima, and interpretations.

It is possible to construct other audio-conditioning systems that are simpler: normalize mel-band energies and sample directly from them, use a Dirichlet distribution over motifs, run a weighted choice over theme embeddings, or inject a fixed noise schedule into a language model. The value of a three-qubit statevector here is pedagogical and creative rather than computational necessity. It provides a compact, explicit series of nonlinear mixing and interference operations whose outputs can be inspected independently. Since the state space has only eight entries, NumPy evaluates it trivially on classical hardware. Any claim of improved efficiency or genuinely quantum randomness would require a fundamentally different experiment.

7 Quantum-style circuit construction and equations

The circuit begins in the computational-basis state ¿, represented by the length-eight complex vector $(1, 0, 0, 0, 0, 0, 0, 0)^T$. The three qubits are indexed $q = 0, 1, 2$, with qubit zero used as the most significant binary digit. In source code, a single-qubit gate on a chosen wire is embedded in the full system by Kronecker products with two identity matrices. Thus a gate on the middle wire has the full matrix $I \otimes U \otimes I$. The code uses np.kron to build these 8×8 operators and ordinary matrix-vector multiplication to update the state.

The Hadamard gate is

$$H=\frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\ 1 & -1\end{bmatrix},$$

and the parameterized rotations are

$$R_Y(\theta)=\begin{bmatrix}\cos(\theta/2) & -\sin(\theta/2)\\ \sin(\theta/2) & \cos(\theta/2)\end{bmatrix}, R_Z(\phi)=\begin{bmatrix}e^{-i\phi/2} & 0\\ 0 & e^{i\phi/2}\end{bmatrix}.$$

All three operations are unitary, preserving the state vector's Euclidean norm up to floating-point error. The Hadamard creates superposition, R_Y redistributes amplitudes, and R_Z introduces relative phase. Phase can influence the ultimate probabilities when followed by noncommuting mixing operations, which makes this construction more expressive than assigning each image a fixed real-valued weight by hand.

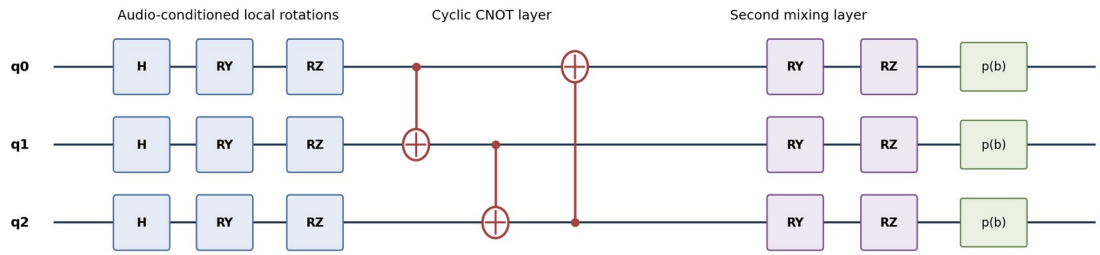


Figure 3. Schematic of the actual gate ordering. Each wire receives Hadamard, RY, and RZ; three successive cyclic CNOT gates connect the wires; a final pair of rotations precedes probability extraction. A schematic is not a hardware execution trace.

The first-stage Y-rotation angles are computed from the audio ratios by

$$\theta_q=\pi\left(0.2+1.6\sqrt{r_q}\right), q=0,1,2.$$

This transformation was manually designed to avoid sending all the small-valued band ratios to angles very near zero. It is not a fitted or optimized transfer function. For the observed track, the three angles were 5.331106, 2.350297, and 1.058160 radians. On each wire, the circuit applied Hadamard, then $R_Y(\theta_q)$, then an R_Z whose angle uses the *next* band in cyclic order: $\phi_q=\pi r_{(q+1)\bmod 3}$. The cyclic indexing deliberately mixes the input dimensions in the local phase preparation.

After this local layer, the program applies three controlled-NOT operations in the order $CNOT_{0\to 1}$, $CNOT_{1\to 2}$, and $CNOT_{2\to 0}$. A controlled-NOT gate flips its target qubit whenever its control qubit is one; the program implements it directly by assigning ones to the appropriate entries of a permutation matrix. This cyclic sequence couples all three wires. The gate pattern permits entanglement, but this project did not calculate an entanglement monotone for the resulting state or experimentally detect entanglement. Calling it an "entangling layer" describes the type and arrangement of gates, not a separately measured empirical result.

A second mixing layer follows. On wire q, the program applies $R_Y\left(0.47\,\theta_{(q+1)\bmod 3}\right)$ and then $R_Z\left(0.37\,\theta_{(q+2)\bmod 3}\right)$. Constants 0.47 and 0.37 are hand-set. Their values matter to exact reproduction but were not selected through a search over songwriting outcomes, nor justified as physically meaningful parameters. In functional order, if G_1 denotes the first local gates, C the three CNOTs, and G_2 the final local gates, then $|\psi_f\rangle = G_2 C G_1 |000\rangle$. Readers should pay attention to matrix multiplication order: the rightmost factor acts first, and the program updates the state after each operation rather than constructing the full product as a single symbolic expression.

Computational-basis measurement outcomes are the strings $b \in \{000, 001, 010, 011, 100, 101, 110, 111\}$. The probability assigned to each is given by the Born rule:

$$p_b = |\langle b|\psi_f\rangle¿^2, \sum_b p_b = 1.$$

In the original implementation this is p = np.abs(state)**2; p = p/p.sum(). The final division provides numerical normalization. The state itself is still a pure state: its density matrix $\rho = |\psi_f\rangle\langle\psi_f|$ has $Tr(\rho^2) \approx 1$ and von Neumann entropy $-Tr(\rho \log_2 \rho) = 0$ in ideal exact arithmetic. The reported **nonzero** entropy is instead the Shannon entropy of measurement *outcomes in one chosen basis*. The distinction matters, because calling 1.622796 bits "the quantum state's entropy" without qualification would confuse measurement uncertainty with mixed-state thermodynamic or von Neumann entropy.

Finally, the statevector was not repeatedly collapsed in the simulator to obtain physical quantum shots. The code calculated probabilities once, initialized np.random.default_rng(seed), and drew 4096 integers with rng.choice(8, 4096, p=prob). In hardware language these were synthetic "shots"; in implementation terms they were independent pseudorandom categorical selections from an already calculated array. Because the signal-conditioned seed and probability array are reproducible, the motif ranking is reproducible for the file and software configuration. This is precisely the limited relationship between the quantum-style mathematical circuit and the classical creative system.

8 Measurement entropy and the recorded numerical results

For a finite distribution $p = (p_0, \ldots, p_7)$, Shannon entropy in bits is $H(p) = -\sum_{b=0}^{7} p_b \log_2 p_b$, adopting $0 \log_2 0 = 0$. The maximum possible entropy over eight outcomes is $\log_2 8 = 3$ bits; it occurs for a uniform distribution in which all outcomes are equally likely. The minimum is zero when one outcome has probability one. Shannon's 1948 theory provides the information-theoretic foundation for interpreting entropy as average uncertainty in a discrete variable [9]. That interpretation does not require the variable to originate from a physical quantum process.

The actual output probabilities, rounded to six decimal places, were as follows. The rounded entries add to approximately one; full-precision values were used in calculating the entropy.

Basis outcome	Probability	Assigned motif	Rhyme palette
000	0.137610	streetlights	night / light / bright
001	0.004581	rain on glass	rain / again / remain
010	0.005387	match flame	fire / higher / wire
011	0.661529	neon skyline	glow / slow / go
100	0.012741	engine hum	drive / alive / survive
101	0.014463	open doorway	door / more / before
110	0.089968	rising smoke	smoke / broke / woke
111	0.073720	first light	day / away / stay

Substitution into the Shannon sum gives $H = 1.622796$ bits. Division by the maximum of 3 bits yields $H_{norm} = 1.622796/3 = 0.540932$, approximately 54.09 percent. That percentage is the *fraction of maximal entropy for an eight-category distribution*, not a score stating that the lyrics are 54 percent cohesive or 54 percent novel. A second descriptive metric, min-entropy $H_{\infty} = -\log_2 max_b\, p_b$, is approximately 0.596 bits because outcome 011 dominates. The effective number of equally likely categories under Shannon's measure, 2^H, is approximately 3.08. These complementary views indicate a distribution that is neither uniform nor fully concentrated: it strongly favors one outcome while preserving lower-probability alternatives.

The histogram produced by 4,096 synthetic draws selected 011 on 2,697 occasions, 000 on 567, 110 on 379, and 111 on 308. These are the four most frequent outcomes, not the whole histogram. Their count ordering matches the leading four exact theoretical probabilities. For instance, 4096×0.661529 is approximately 2,710, close to but not identical to the observed 2,697. Sampling deviations are expected even when the code works correctly. The motif-selector function sorted outcomes by synthetic count and chose the two highest distinct entries. The primary became "neon skyline"; the secondary became "streetlights." It also took the final sampled three-bit value as an index into the action-word bank and reported the word "return." The final creative draft did not mechanically insert that word everywhere; rather, the larger story emphasized movement from restriction toward self-possession.

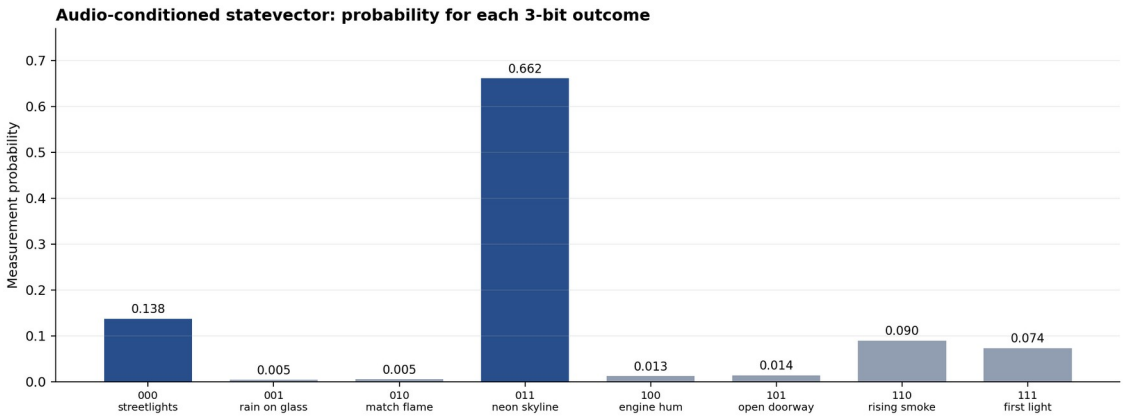


Figure 2. All eight computed Born-rule measurement probabilities for the three-qubit statevector. The dominant outcome 011 maps to neon skyline; 000 maps to streetlights.

The histogram and probability table describe **one input file under one fixed circuit**. They provide no estimate of how much the output would change across songs, across genres, or across slight edits of the same mix. Nor do they establish that higher measured entropy would produce better writing. In principle, a distribution with three bits of entropy could encourage unfocused movement across eight unrelated motifs, while a low-entropy distribution could force too much repetition. The useful creative variable may instead be the interaction of *limited variety* with *strong structural constraints*. That hypothesis motivates the explicit rules in the next section.

A reproducibility check was performed by invoking the original quantum_rgb_entropy.py on the same WAV and writing a new JSON file. The new and saved JSON outputs compared byte-for-byte without differences. This confirms the calculations, seed, motif ranking, and selected rhyme families in the tested runtime. It does not, by itself, authenticate the unlogged subjective decisions used to draft the song text, and it does not validate the physical realism of the circuit as a proposed songwriting aid.

9 From probability outcomes to cohesive lyric constraints

The motif bank was intentionally narrow and semantically curated before the pseudorandom selection. Its eight entries were streetlights, rain on glass, match flame, neon skyline, engine hum, open doorway, rising smoke, and first light. These are not random words sampled from the whole dictionary; each belongs to a compatible world of night travel, illumination, transition, or release. Likewise, the associated rhyme banks were deliberately hand-built groups with some shared phonetic or narrative affordance. The existence of this prepared semantic bank substantially explains why the output was usable. Sampling controls *which* compatible ideas lead, while editorial design controls *what kinds* of ideas may be sampled at all.

A formal description helps separate the moving parts. Let A be the audio, $f(A)$ the three-band feature extractor, $Q(f(A))$ the circuit-produced measurement distribution, and $S(Q,\xi)$ the seeded selection operator using pseudorandom state ξ. The selected motif pair is $M = S(Q(f(A)),\xi)$. Then let C denote the manually chosen constraints: a single dramatic

trajectory, at most two principal recurring motifs, stable chorus wording, a selected rhyme vocabulary, and approximately one lyric line per bar. The finished lyric is better described as $L = D(M, C, A, P)$, where D is the interpretive drafting process and P is the vocal-performance brief. The experiment does not implement D as a callable deterministic program, but it records its outputs and major inputs.

This decomposition illustrates a reproducibility limit and a genuine design insight. The machine can reliably produce a palette, but a narrative writer still needs to judge whether the phrase "streetlights go low" fits the song's emotional goal, whether a verse advances the story rather than merely decorating it, and whether the word "glow" can carry a sustained vowel at the end of a hook line. The process is therefore a collaborative creative workflow, not a claim that stochastic circuit measurement is a language-generation algorithm. One could build a fully automated decoder later, but doing so would introduce other choices—such as rhyme scoring, semantic similarity, meter constraints, and generation temperature—that were absent from the current implementation.

The selected imagery establishes a spatial and emotional continuity. The first verse begins with "Neon on an empty street," setting the image world directly. Later lines turn the skyline into a constant against the narrator's confusion and use traffic lights, windows, and roads to extend the same metaphor. The first pre-chorus speaks of "the city breathing below," then invokes a waking streetlight and multiple answering lights. The chorus states "BRING ON THE NIGHT / I'VE GOT MY OWN LIGHT," resolving the contrast between an external environment and an internal source of direction. The closing line "I've got my own glow" reappears at the end, so the conclusion is not the arbitrary arrival of a new symbol; it is a recapitulation of one already introduced.

Rhyme provides another continuity mechanism. The 011 palette contributes glow, slow, and go, while the 000 palette contributes night, light, and bright. The hook relies particularly on NIGHT/LIGHT and LOW/GLOW/GO, the latter including a near-family relation rather than treating all endings as one rigid three-word list. This makes the chorus easy to remember and leaves space for a long stressed vowel on "night," "light," and "glow." The remaining lyric uses additional local rhymes—street/me, wait/gate, turn/learn, grip/slip, black/back—to keep verses natural. In other words, the entropy-selected palette acted as a **global anchor**, not a rule that every pair of lines must rhyme with one of six preselected words.

Narrative cohesion was also manually imposed at the level of causal change. The early lines depict waiting for external approval and repeating the same road or choice. The middle of the verse marks the recognition of an internal voice. The pre-chorus enacts release, and the chorus speaks from a newly claimed position of control. The bridge avoids another full repetition of the initial conflict by using "I don't have to call you back," a behavior consistent with the resolution. The outro revisits the light imagery but with lower performance intensity, fitting a transition from triumph toward calm. This structure gives the lyric a beginning, change, and resolution rather than a montage of visually related lines.

The stored cohesion configuration makes three of these rules explicit: story_arc: trapped -> resistance -> release -> renewal, motif_limit: 2, and keep_hook_identical_each_repeat: True. These are hard-coded design intentions, not weights learned from a corpus. The repeated chorus was in fact kept the same in its full eight-bar appearances. The four-bar last reprise and

the outro intentionally compress or reinterpret that material, which should not be mistaken for an accidental failure to obey the identical-repetition guideline.

10 Prosody, rhyme cadence, and the Suno-facing specification

For 4/4 at 133.928571 BPM, a full measure supplies approximately 1.792 seconds. That time is sufficient for roughly eight eighth-note positions, represented as 1, ¿, 2, ¿, 3, ¿, 4, ¿, but it is not enough to guarantee that any arbitrary eight-syllable line will sound smooth. English prosody contains stressed and unstressed syllables, reduced vowels, consonant clusters, and phrase-final elongation. A bar assignment is therefore an initial rhythmic envelope; local syllable timing still must accommodate speech stress and the contour of the melody.

The verse was written as a relatively close, syncopated sequence. Lines such as “Neon on an empty street” and “Footsteps keeping time with me” are compact enough to receive short note values with a little space near the end. The recommended delivery places clear accents on strong semantic syllables and avoids forcing every grammatical connector onto a heavy downbeat. Verse lines generally contain about six to eight spoken syllables, not exactly eight each; “So I loosened up my grip” and “And I’m choosing my own side” demonstrate why an exact syllable count should be checked rather than assumed from the printed line length. Where a line has fewer syllables, the difference becomes a rest, a pickup, or a held end word.

The pre-chorus deliberately shifts from verbal density to anticipation. “I used to wait” can occupy the opening portion of one bar while leaving a breath or instrumental answer. “For your green light to go” adds motion without discarding the light metaphor. The last two lines of the unit—“Three, two, one - let go / Watch me come alive”—are meant to approach the full hook. The countdown provides a singable rhythmic cue and can be synchronized to a drum fill or rising synth. At the start of the chorus, strong onsets should align with large-scale metric stress, but placing the phrase *exactly* on a particular drum transient requires listening and, ideally, a vocal guide recorded against the beat.

The chorus uses four successive one-bar vocal gestures: “BRING ON THE NIGHT,” “I’VE GOT MY OWN LIGHT,” “WHEN THE STREETLIGHTS GO LOW,” and “I’VE STILL GOT THAT GLOW.” A possible first-line design stresses BRING on beat one and NIGHT by the third beat, retaining beat four or its latter subdivision as a sustained note. The next line emphasizes OWN and LIGHT, and the following lines create a repeated low/glow tail. This is an *illustrative cadence*, not a published MIDI transcription. Because Suno can reinterpret phrasing, the final user document explicitly recommends simplifying crowded lines—for example, changing “WHEN THE STREETLIGHTS GO LOW” to “WHEN THE LIGHTS GO LOW”—if a generated vocal rushes the consonants.

Doubling and register are part of the compositional instruction, not measured facts about the backing track. The proposed Suno style asks for a close lead in the verse, rising intensity in pre-choruses, doubled and harmonized choruses, selective crowd-like support on the hook, and a more spacious breakdown. Such contrast is plausibly supported by the energy map, but a generation engine may not faithfully obey each tag. In particular, the use of [Verse 1], [Pre-Chorus], and [Chorus] in a custom lyric field is a prompt convention; it is not equivalent to controlling the exact internal arrangement of a text-to-music model. The writer should

audition outputs, correct section mismatch, and treat written timestamps as targets rather than promises.

The lyric itself omits every term whose sole role belongs to the computational apparatus. “Qubit,” “entropy,” “Born rule,” “spectrogram,” “rotation gate,” and “NumPy” are explanatory vocabulary for the producer’s workflow, not lyrical metaphors. That design satisfies the original request to let mathematical selection influence vocabulary without turning the song into an instructional rap about mathematics. It also makes the creative effect of the circuit harder to identify by casual listening, which is exactly why scientific evaluation would need controlled comparison rather than asking whether listeners can detect a quantum reference.

11 Exact recorded prompts, files, and reproduction commands

The user-facing request establishing the method was: “anyalize this via spectrogram then build lyrics for it please with instructions etc how to cadence the rhymes for Suno build quantum rgb circuit for entropy for using numpy inspired by Pennylane then don’t mention that in the lyrics but use the entropy for better lyrical generation and cohesion please.” The unusual spelling is preserved here because the point is provenance, not copyediting. This recorded request set the analysis, implementation, songwriting, and exclusion constraints. It did not specify gate architecture, fixed energy bands, a random seed, or the song’s eventual title; those choices were made during implementation and drafting.

The exact stored **Suno Style of Music** text was:

> Driving minor-key alternative electro-pop with anthemic synth-rock impact, 134 BPM, tight pulsing sub-bass, crisp four-on-the-floor energy, bright rhythmic synth accents, intimate syncopated lead vocal in the verse, rising urgent pre-chorus, explosive catchy doubled chorus with a strong melodic hook, emotional but triumphant, wide harmonies on held end-rhymes, selective crowd-shout doubles on the hook, spacious half-time breakdown at the bridge, final quiet glow and reverb tail. Avoid overbusy vocal runs. Clear diction. Preserve the existing instrumental structure if using uploaded audio.

The exact final **Suno Lyrics** field is retained, with all section labels and lyric lines, in the separate companion file Bring_On_The_Night_Suno_Lyrics_and_Cadence.md. That file is part of the reproducibility archive. Reprinting more than one hundred lyric lines in the main methods body would add pages without improving the derivation of the feature extractor or the entropy circuit, so this paper examines representative excerpts and provides the full original text as primary-source supplementary material. The original Python implementations are likewise distributed as exact source files rather than silently rewritten to match the prose descriptions.

To reproduce the spectrogram and output JSON in an environment where the audio file and paths exist, run python analyze_audio.py. Because the original script uses absolute /mnt/data/ paths, either place the WAV at the exact named location or edit the path string before execution. The script generates track_spectrogram.png and track_analysis.json. To

reproduce the entropy output, use the following shell command; quoted filename handling matters because the source WAV path contains spaces and parentheses:

```
python quantum_rgb_entropy.py \
 'Punchier Pre-Chorus and Chorus (Extended) (1) (1) (1).wav' \
 --json quantum_rgb_entropy_results.json
```

A portable second-stage analysis could instead provide --input and --output-dir flags to both scripts, save an environment lockfile, and write both precision-level arrays and source hashes to a machine-readable manifest. These would improve the software engineering but were not part of the original audio-analysis run. Similarly, the exact equations and code here are verified from the saved implementation; no claim is made that the final lyrical text emerged through a deterministic call to the circuit code or that an automatic lyric evaluator was executed.

For readers who want to reproduce the *creative drafting logic* with another capable text generator, the following is a **proposed reconstruction prompt**, not a recovered hidden historical prompt:

> Write an alternative electro-pop song for an approximately 134 BPM, 4/4 instrumental of 162.8 seconds. The central narrative is restriction, resistance, release, and renewal. Use neon skyline and streetlights as the two primary recurring motifs; favor the rhyme families glow/slow/go and night/light/bright. Maintain one coherent nighttime-city world, keep the chorus wording stable between full repeats, use a close rhythmic verse, a sparse and rising pre-chorus, and a short declarative singable hook. Aim for approximately one lyric line per bar, allowing long vowels at line endings and more space during breakdowns. Supply section labels and vocal-performance instructions for Suno. Do not mention quantum computing, circuit gates, entropy, source code, or signal-processing tools in the lyrics.

This reconstructed prompt states the semantic constraints as plainly as possible and could serve as the control specification for future benchmarking. Because the original final writing involved ordinary linguistic judgment, a replication can legitimately produce a different lyric with the same motif and rhythmic properties. That variation should be reported as an expected property of the language generation stage, not confused with a failure to reproduce the quantum-style probability table.

12 Limitations, validity threats, and responsible interpretation

Several limitations constrain any scientific claim about the system. The experiment uses one song and one completed lyric, and it contains no blinded quality ratings, matched baseline conditions, independent producers, or repeated lyric-generation trials. The probability distribution has a mathematically verifiable entropy, but the quality and cohesion of the final prose are not measurements on that distribution. The original eight-item motif bank was carefully curated to be mutually compatible, so any apparent improvement may arise from the bank, the story outline, the writer's skill, or the repeated chorus rather than the quantum-style mixing. These are not negligible confounds: they are central to what makes a useful constrained-writing workflow.

Signal-processing decisions introduce their own sensitivity. Stereo downmixing may cancel phase-inverted content; 12 kHz resampling excludes information above 6 kHz; the chosen 8,192-sample FFT frame and three frequency bands affect the normalized RGB vector; the beat tracker may return a double-time or half-time interpretation on a different track; and the tonal score comes from whole-track averaged chroma rather than independently annotated harmony. A visual rise in energy may mark a transition without confirming its formal role. Finally, the per-bar lyric map was generated from a nominal 4/4 grid, not verified syllable-by-syllable against produced vocals. Every one of these limits is correctable in principle, but none should be retrospectively described as already solved.

On the circuit side, the small statevector is not computationally demanding, and the rotation coefficients are untrained. Its Born-rule probabilities are deterministic functions of the audio features; NumPy sampling adds pseudorandom variability conditional on a deterministic seed. The histogram does not prove access to physical entropy, and the calculation of measurement Shannon entropy should not be confused with the von Neumann entropy of the pure state. Describing the method precisely safeguards readers against an exaggerated impression of quantum advantage. Equally important, the use of entropy is not intrinsically harmful or helpful to creativity. High uncertainty can lead to productive variation or to unfocused text, depending on the surrounding constraints.

13 Proposed experimental evaluation and ablation study

A stronger study would collect a diverse licensed corpus of instrumental excerpts with validated tempo and downbeat grids. Each excerpt would be assigned the same narrative brief and target lyric length across at least four conditions: a manually curated palette without random selection; direct sampling from normalized audio-band energies; classical seeded categorical sampling from a fixed, non-circuit probability model; and the current audio-conditioned statevector method. The creative generator, instruction length, output budget, and editing time would be held constant. Multiple random seeds should be used within each condition except in a strict single-seed reproducibility test. The study should also compare the effects of switching off the two-motif limit and of switching off the repeated-hook constraint.

Evaluation should separate musical fit from text quality. For prosody, trained annotators could label stressed syllables, phrase-final lengthening, and out-of-bar crowding after vocals are sung or synthesized. For narrative coherence, blinded listeners could rate causal progression, consistency of perspective, image reuse, and whether the ending resolves the opening situation. Distinct judges should assess novelty, emotional impact, memorability, and natural diction. An automated token-based rhyme measure could quantify terminal phonetic similarity, but it should supplement human judgment rather than define what makes a good hook. Inter-rater agreement, statistical uncertainty, and prespecified hypotheses are necessary if the outcomes are to support comparisons among methods.

The most revealing outcome may be an interaction rather than a main effect: perhaps the statevector's moderate output uncertainty only performs well when motif number is limited, while high-entropy settings fail without a coherent scene. This can be investigated by deliberately changing the circuit parameters and measuring the resulting entropy and image diversity. It would also be informative to test whether the same probabilities with randomly

permuted motif labels change judgments. If outcomes remain similar under such permutations, the benefit may lie primarily in the fixed semantic bank. If they worsen, the mapping between favored states and useful motifs may be central. In either case, the conclusion would be empirical rather than based solely on the appeal of the mathematical language.

14 Conclusion

This case documents an integrated but intentionally modest creative pipeline: audio is converted into a spectrogram and rhythmic cues; three frequency-band ratios parameterize a small, classically simulated quantum-style circuit; its eight measurement probabilities provide a reproducible Shannon entropy and seeded motif ranking; and those ranked motifs are incorporated into a narrative lyric with an explicit cadence guide for Suno. The original file supports the observed tempo estimate, tonal candidate, visual arrangement contrasts, RGB ratio, all eight circuit probabilities, and 1.622796-bit entropy. The source code and prompt records are preserved so the technique can be audited, adapted, and challenged. What remains unproven is a causal advantage in lyric quality. The credible contribution is not magical "quantum songwriting," but a transparent method for using quantitative variation inside deliberate artistic constraints.

15 References

[1] PennyLane. "Quantum circuits." Software documentation. https://docs.pennylane.ai/en/stable/introduction/circuits.html. Accessed October 2026.

[2] SciPy. "scipy.signal.resample_poly." API documentation. https://scipy.github.io/devdocs/reference/generated/scipy.signal.resample_poly.html. Accessed October 2026.

[3] SciPy. "scipy.signal.stft." API documentation; notes on STFT frames, overlapping windows, and ShortTimeFFT as the newer interface. https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.stft.html. Accessed October 2026.

[4] librosa. "librosa.onset.onset_strength." Version 0.11.0 API documentation. https://librosa.org/doc/0.11.0/generated/librosa.onset.onset_strength.html. Accessed October 2026.

[5] mir_eval. "Structural segmentation evaluation." Version 0.8.2 API documentation. https://mir-eval.readthedocs.io/latest/api/segment.html. Accessed October 2026.

[6] Ellis, D. P. W. "Beat Tracking by Dynamic Programming." *Journal of New Music Research* 36, no. 1 (2007): 51–60. The implementation is also described in https://librosa.org/doc-playground/latest/generated/librosa.beat.beat_track.html.

[7] librosa. "librosa.feature.chroma_stft." API documentation. https://librosa.org/doc/main/api/generated/librosa.feature.chroma_stft.html. Accessed October 2026.

[8] Krumhansl, C. L., and E. J. Kessler. “Tracing the Dynamic Changes in Perceived Tonal Organization in a Spatial Representation of Musical Keys.” *Psychological Review* 89, no. 4 (1982): 334–368. https://doi.org/10.1037/0033-295X.89.4.334.

[9] Shannon, C. E. “A Mathematical Theory of Communication.” *Bell System Technical Journal* 27 (1948): 379–423 and 623–656. https://doi.org/10.1002/j.1538-7305.1948.tb01338.x; https://doi.org/10.1002/j.1538-7305.1948.tb00917.x.

[10] NumPy. “Random sampling” and numpy.random.default_rng. API documentation. https://numpy.org/doc/2.1/reference/random/. Accessed October 2026.

16 Supplementary materials and executable archive

S1 — Original analysis source (analyze_audio.py). Stereo input, downmix, resampling, short-time Fourier transform, spectrogram plotting, onset envelope, beat estimation, chroma key correlation, eight-second section metrics, and output JSON. The script as provided has fixed /mnt/data/ paths; the code has not been altered in the archive to disguise this assumption.

S2 — Original circuit source (quantum_rgb_entropy.py). Exact NumPy implementation of Hadamard, RY, RZ, Kronecker placement, three controlled-NOT gates, audio-conditioning, statevector evolution, probability calculation, Shannon entropy, pseudorandom synthetic sampling, and motif-bank selection. It imports SoundFile and SciPy for audio input and resampling but not PennyLane for quantum simulation.

S3 — Observational results (track_analysis.json, quantum_rgb_entropy_results.json). Full machine-readable feature summaries, key candidates, tempo estimates, probabilities, angles, seed, and imagery selections. S1 and S2 are the source of these records, not reverse-engineered approximations created for this paper.

S4 — Visuals (track_spectrogram.png, measurement_probabilities.png). Original spectrogram plus an explanatory probability chart created for this research paper. The latter is a visualization of existing numerical results, not a new experiment.

S5 — Final writing artifact (Bring_On_The_Night_Suno_Lyrics_and_Cadence.md). Exact saved song title, Suno Style field, full Lyrics field with tags, timestamp map, breath and accent guidance, and note explaining the computational palette. Together the supplements define which portions of the pipeline can be independently executed and which depend on creative textual judgment.